

PROJECT DESIGN PHASE – II

Technology Stack (Architecture & Stack)

Date: 30 September 2026

1. Project Details:

Team ID	SWTID-2026-3350
Project Name	PocketSmart AI – Your Smart Budget & Recommendation Assistant
Team size / Members	4 – Esther Ammo M , Epsipa R , Eswari M , Fayisa Fathima A

2. Technical Architecture

The PocketSmart AI architecture separates the presentation layer, application logic, input validation, financial data processing, database access, AI recommendation services, and external interfaces. The design supports budget tracking, expense analysis, personalized recommendations and AI-assisted financial guidance.

3. Table-1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Interface through which users enter income, expenses, budgets and preferences and view recommendations.	HTML, CSS, Jinja2 Templates
2	Application Logic-1	Receives and processes user financial information, budget records and recommendation requests.	Python + FastAPI
3	Application Logic-2	Validates request data, expense values, categories, budget inputs and API schemas.	Pydantic
4	Application Logic-3	Provides database access and object-relational mapping for users, budgets, expenses and recommendations.	SQLAlchemy
5	Database	Stores user details, income, expenses, budgets, categories and recommendation-related records.	SQLite

6	Cloud Database	Cloud database service for the current project architecture.	Not specified / Not used in current local architecture
7	File Storage	Stores application assets and locally required project files.	Local Filesystem
8	External API-1	External interface used to request AI-generated budgeting insights and personalized recommendations.	Google Gemini API
9	External API-2	Additional external financial-data API in the current solution.	Not specified / Not used
10	Machine Learning / AI Model	Analyzes user-provided financial context and generates personalized budgeting insights, recommendations and explanatory responses.	Google Gemini Models – Gemini Pro / Gemini Flash
11	Infrastructure (Server / Cloud)	Application deployment and runtime environment for the backend service.	Local Server / Uvicorn; cloud deployment not specified

4. Infrastructure Demarcation:

Area	Components	Purpose
Local	Browser, Jinja2, FastAPI, Pydantic, SQLAlchemy, SQLite, local filesystem	User interface, business logic, validation, budgeting operations and local data storage.
External / Cloud	Google Gemini API and Gemini models	AI-powered budget analysis, personalized recommendations and natural-language financial assistance.

5. Table-2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Frameworks and libraries used to build the application.	FastAPI, Pydantic, SQLAlchemy, Jinja2
2	Security	API credentials are kept in environment configuration and should not be exposed in source code or the user interface. Input validation is applied to financial and user-submitted data.	Environment variables / secure API-key configuration; Pydantic validation

3	Scalable Architecture	The application separates presentation, backend logic, AI integration and database access, allowing individual components to be extended.	FastAPI + Jinja2 + SQLAlchemy + modular Python structure
4	Availability	The current project is designed to run while the configured local Uvicorn service is active. Distributed servers and load balancers are not specified in the current solution.	Uvicorn / local server
5	Performance	FastAPI provides lightweight request handling; Gemini model selection can separate detailed recommendation workloads from concise assistance; SQLite supports the current local project scope.	FastAPI + Gemini Pro/Flash + SQLite

6. Technology Stack Summary

- Frontend: HTML, CSS and Jinja2 templates
- Backend: Python and FastAPI
- Validation: Pydantic
- Database layer: SQLAlchemy
- Database: SQLite
- AI / Machine Learning: Google Gemini models
- External interface: Google Gemini API
- Local server: Uvicorn
- File storage: Local filesystem

7. Technology Roles in PocketSmart AI

Technology	Role in PocketSmart AI
HTML, CSS, Jinja2	Builds the user-facing screens for entering financial information and displaying budgets, expense summaries and recommendations.

Python + FastAPI	Implements APIs, application workflows and backend business logic.
Pydantic	Validates user inputs and API request/response structures.
SQLAlchemy	Connects the application logic with the database through ORM models.
SQLite	Provides lightweight local persistence for user and financial records.
Google Gemini API	Connects PocketSmart AI to the external generative-AI service.
Gemini Models	Generates natural-language budget insights, recommendations and explanations from the permitted user context.
Uvicorn	Runs the FastAPI application locally.

8. References / Architecture Guidance

- C4 Model – <https://c4model.com/>
- IBM Cloud Architecture – <https://www.ibm.com/cloud/architecture>
- AWS Architecture – <https://aws.amazon.com/architecture>